

This is an excellent question.
This is exactly the transition from:
“developer setup”
to
“production deployment mindset”

The Core Idea

Instead of:

Airflow DAG
↓
git pull
↓
run latest code

You do:

Developer pushes code
↓
CI/CD pipeline builds deployment
↓
Server/container updated
↓
Airflow runs already-deployed stable code

This ensures:

- reproducibility
 - version consistency
 - rollback capability
 - safer deployments
-

Professional Deployment Flow

Recommended Architecture

GitHub
↓
GitHub Actions (CI/CD)
↓
Build Docker image
↓
Push image to registry
↓
Server pulls latest image
↓
Restart Airflow containers

Why this is better

Problem with git pull during DAG execution

Imagine:

- DAG starts at 10:00
- another developer pushes code at 10:01
- git pull happens mid-run

Now:

- some tasks run old code
- some tasks run new code

This creates:

- inconsistent pipelines
 - debugging nightmares
-

Correct Professional Pattern

Airflow should run:

- immutable deployed code
- fixed image version

NOT changing live code.

Recommended Stack for YOU

Since you're solo:

SIMPLE + PROFESSIONAL

Use:

- GitHub
- GitHub Actions
- Docker Compose
- EC2

This is enough.

Step-by-Step Deployment Design

STEP 1 — Code pushed to GitHub

git push origin main

STEP 2 — GitHub Actions triggers

GitHub Actions:

- runs tests
 - builds Docker image
 - deploys to server
-

STEP 3 — Build Docker image

Example:

FROM apache/airflow:2.9.0

COPY airflow/ /opt/airflow/

COPY dbt/ /opt/dbt/

RUN pip install dbt-snowflake

Now:

- Airflow
 - DAGs
 - dbt project
- all exist inside image.
-

STEP 4 — Push image to registry

Use:

- Docker Hub
- OR
- Amazon Web Services ECR
-

STEP 5 — Server updates container

On EC2:

docker compose pull

docker compose up -d

Now server runs new stable image.

STEP 6 — Airflow executes deployed version

Airflow now runs:

- tested
- versioned
- immutable code

This is production-grade.

Recommended Folder Structure

```
project/
|
|— airflow/
|   |— dags/
|   |— plugins/
|
|— dbt/
|
|— docker/
|   |— Dockerfile
|
|— .github/
|   |— workflows/
|
|— docker-compose.yml
```

Example GitHub Actions Flow

.github/workflows/deploy.yml

name: Deploy

on:

push:

branches:

- main

jobs:

deploy:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Build Docker image

run: docker build -t my-airflow .

- name: Deploy to EC2

run: ssh ec2-user@server "docker compose up -d"

Senior-Level Deployment Principle

Airflow is NOT your deployment tool

Airflow is:

- orchestration engine

GitHub Actions / CI-CD is:

- deployment system

This separation is extremely important.

Another Important Principle

“Build once, run everywhere”

Same Docker image should run:

- locally
- staging
- production

Only configuration changes.

This is a core DevOps principle.

Professional Deployment Levels

Level 1 (Your ideal current stage)

GitHub Actions

↓

Docker Compose

↓

EC2

Very good already.

Level 2

ECS / Kubernetes

Only when scale requires.

Very Important Operational Advice

NEVER edit production directly

Avoid:

- SSH into server and changing DAGs manually

Always:

- update via Git
- deploy through pipeline

This prevents configuration drift.

What experienced engineers care about

They ask:

- Can I rollback quickly?
- Can I reproduce yesterday's run?
- Can I know which version produced this table?
- Can deployment fail safely?

Your deployment strategy should answer YES.

BEST PRACTICAL START FOR YOU

I strongly recommend:

Phase 1

- Docker Compose
- GitHub
- GitHub Actions
- EC2

This is already professional.

What I'd personally recommend for your exact case

Your ideal architecture

GitHub
↓
GitHub Actions
↓
Docker Build
↓
EC2 Docker Compose
↓
Airflow + dbt
↓
Snowflake

This is:

- modern
- maintainable
- realistic
- production-capable

without unnecessary complexity.

Next, I can help you build:

1. Full production docker-compose.yml
2. Production Dockerfile
3. GitHub Actions deployment pipeline
4. EC2 deployment setup
5. Secure secrets management
6. Airflow + dbt integration
7. Snowflake connection management
8. End-to-end CI/CD architecture

Those are the actual next professional steps.